

Wireframe y Diseño UX/UI - Fitbit Fitness Tracker

Autor: React Frontend Assistant
Proyecto: Fitbit API - Frontend Cliente React + TypeScript
Fecha: Enero 2025
Versión: 1.0

Tabla de Contenidos

- [1. Resumen Ejecutivo](#)
- [2. Arquitectura de Navegación](#)
- [3. Wireframes por Pantalla](#)
- [4. Especificaciones Técnicas](#)
- [5. Mapeo de Componentes React](#)
- [6. Integración con API](#)
- [7. Plan de Implementación](#)

Resumen Ejecutivo

Este documento contiene el diseño completo de la interfaz de usuario (UX/UI) para el frontend del proyecto **Fitbit Fitness Tracker**, una aplicación web desarrollada en React + TypeScript + Vite que consume la API REST Laravel documentada.

Objetivos del Diseño

-  **Moderno y deportivo:** Paleta energética con gradientes vibrantes
-  **Funcional y claro:** Navegación intuitiva según roles de usuario
-  **Modular:** Componentes React reutilizables y escalables
-  **Responsive:** Diseño adaptable a mobile, tablet y desktop
-  **Accesible:** Cumpliendo estándares WCAG 2.1

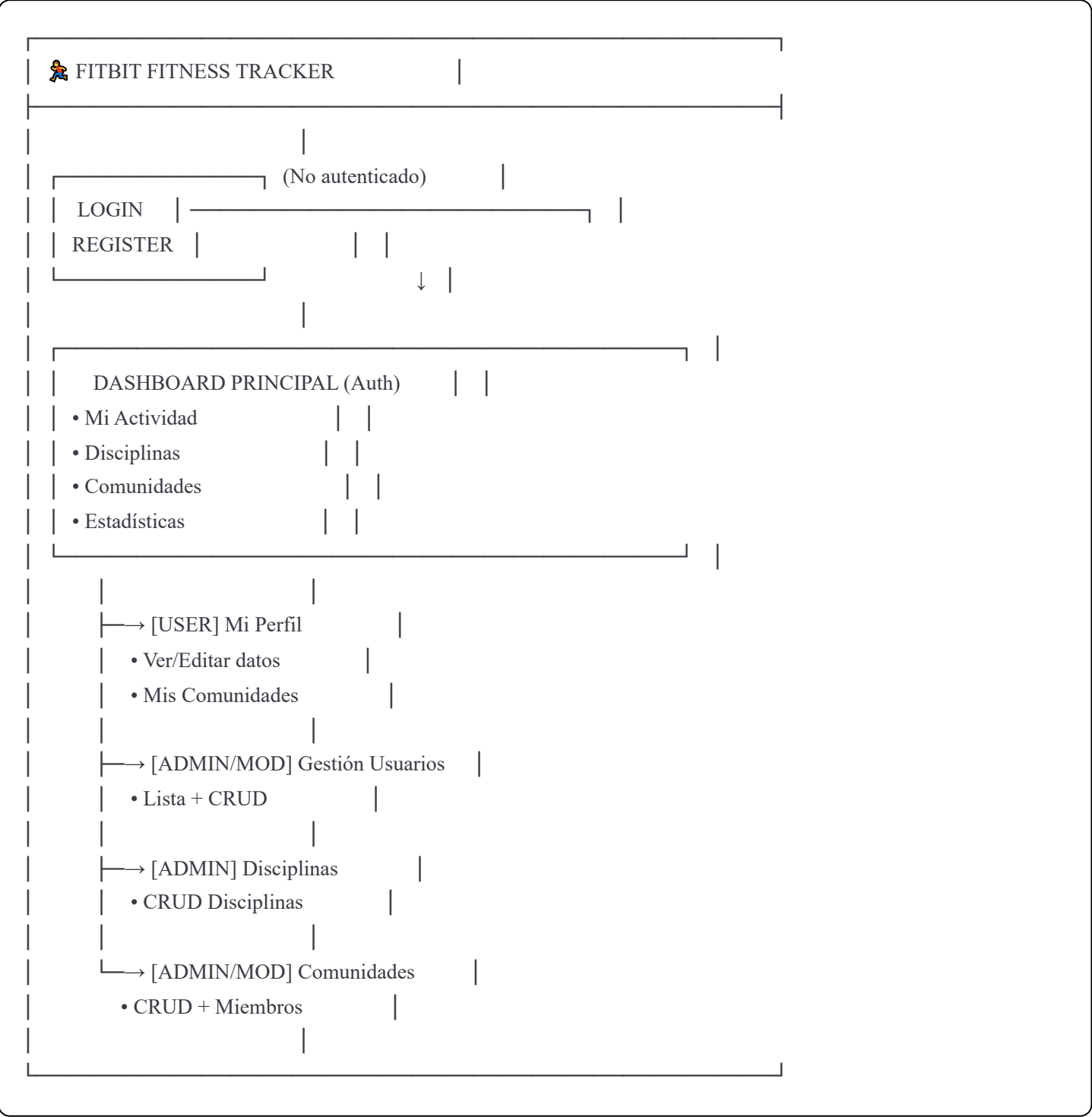
Stack Tecnológico Frontend

- React 18+
- TypeScript 5+
- Vite (bundler)
- TailwindCSS (styling)
- React Router v6 (navegación)
- TanStack Query (data fetching)

- Recharts (visualizaciones)
- Axios (HTTP client)

Arquitectura de Navegación

Mapa de Flujo de Usuario



Rutas de la Aplicación

Ruta	Acceso	Componente	Descripción
<code>/login</code>	Público	<code>LoginPage</code>	Autenticación
<code>/register</code>	Público	<code>RegisterPage</code>	Registro nuevo usuario

FITBIT | Dashboard | Mi Perfil | [Logout] Marc

Hola, Marc Sanchez [Role: User]

MIS ESTADÍSTICAS

COMUNID

DISCIPL

ACTIVOS

3 Unidas

5 Favoritas

142 Miembros

MIS COMUNIDADES

Running Club Barcelona [156 miembros] [Ver]

Cycling Masters [89 miembros] [Ver]

Swimming Team Pro [67 miembros] [Ver]

DISCIPLINAS DISPONIBLES

Running

Cycling

Swimming

Football [+Más]

[+ Unirse a nueva comunidad]

Componentes React:

- DashboardLayout.tsx
- NavBar.tsx
- StatsCards.tsx
- CommunityList.tsx
- DisciplineGrid.tsx
- WelcomeHeader.tsx

Endpoints API:

- GET /api/v1/me
- GET /api/v1/communities
- GET /api/v1/disciplines
- GET /api/v1/stats/communities/summary

Features:

- Visualización de estadísticas personales
- Acceso rápido a comunidades del usuario
- Exploración de disciplinas disponibles
- Acciones contextuales según rol

3 GESTIÓN DE COMUNIDADES (Admin/Moderator)

FITBIT

COMUNIDADES

[+ Nueva Comunidad]

[Buscar comunidad...]

[Filtrar: Todas ▼]

TABLA DE COMUNIDADES

Nombre	Disciplina	Miembros	Acciones
Running BCN	Running	156	
Cycling Masters	Cycling	89	

Swim Team Pro	Swimming	67		
Football League	Football	45		
...	...	...	...	
ESTADÍSTICAS RÁPIDAS				
• Total comunidades: 24				
• Total miembros: 1,240				
• Disciplina más popular: Running (35%)				

Componentes React:

- CommunityTable.tsx
- CommunityForm.tsx (modal)
- SearchBar.tsx
- FilterDropdown.tsx
- QuickStats.tsx
- ActionButtons.tsx

Endpoints API:

- GET /api/v1/communities
- POST /api/v1/communities
- PUT /api/v1/communities/{id}
- DELETE /api/v1/communities/{id}
- GET /api/v1/stats/communities/ranking

Funcionalidades:

- CRUD completo de comunidades
- Búsqueda en tiempo real
- Filtrado por disciplina
- Confirmación de eliminación
- Validación de permisos según rol

4 PERFIL DE USUARIO

--

 FITBIT |  MI PERFIL

MARC SANCHEZ

doom@user.com

[AVATAR] |  Role: User

 Miembro desde: Enero 2025

 INFORMACIÓN PERSONAL

Nombre: [Marc Sanchez] [Editar]

Email: [doom@user.com] [Editar]

Password: [.....] [Cambiar]

 MIS COMUNIDADES (3)

• Running Club Barcelona [Salir]

• Cycling Masters [Salir]

• Swimming Team Pro [Salir]

 Guardar Cambios

 Eliminar Cuenta

Componentes React:

- UserProfile.tsx
- AvatarUpload.tsx
- ProfileForm.tsx
- UserCommunitiesList.tsx
- PasswordChangeModal.tsx

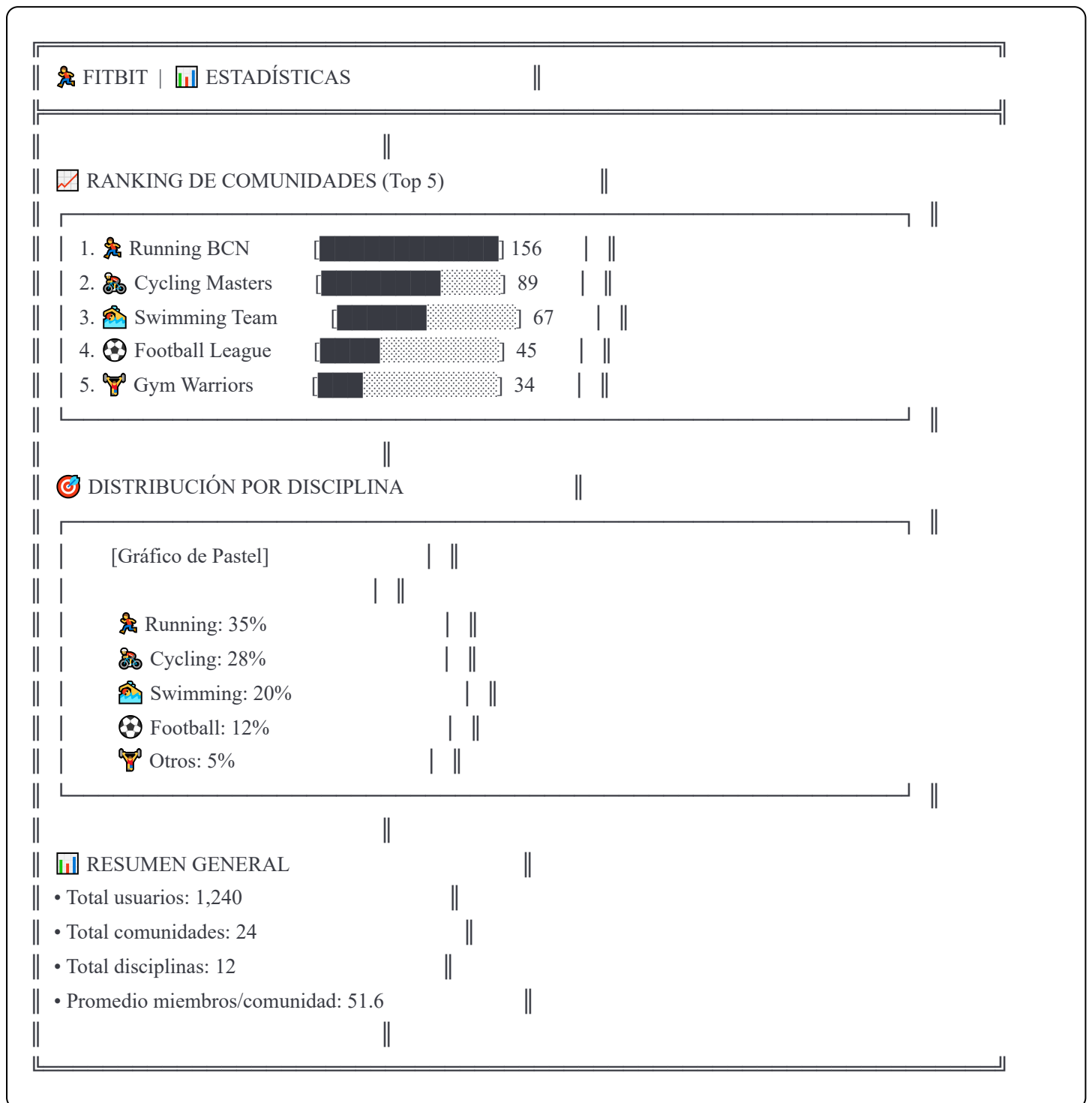
Endpoints API:

- GET /api/v1/users/{id}
- PUT /api/v1/users/{id}
- DELETE /api/v1/users/{id}

Validaciones:

- Edición solo del propio perfil (usuarios normales)
- Admin/Mod pueden editar cualquier perfil
- Confirmación para eliminar cuenta
- Validación de contraseña actual antes de cambiar

5 ESTADÍSTICAS (Admin/Moderator)



Componentes React:

- `StatsLayout.tsx`
- `RankingChart.tsx` (Recharts)

- `DisciplineDistributionChart.tsx` (Recharts)
- `SummaryMetrics.tsx`
- `ExportDataButton.tsx`

Endpoints API:

- `GET /api/v1/stats/communities`
- `GET /api/v1/stats/communities/ranking`
- `GET /api/v1/stats/communities/percentage`
- `GET /api/v1/stats/communities/summary`
- `GET /api/v1/stats/communities/by-discipline`

Features:

- Gráficos interactivos con Recharts
- Actualización en tiempo real
- Exportación de datos (CSV/PDF)
- Filtros por período de tiempo

 Especificaciones Técnicas

 Paleta de Colores

typescript

// theme/colors.ts

```
export const colors = {  
  primary: {  
    gradient: 'linear-gradient(135deg, #667eea 0%, #764ba2 100%)',  
    solid: '#667eea',  
    light: '#a5b4fc',  
    dark: '#5568d3'  
  },  
  secondary: {  
    gradient: 'linear-gradient(135deg, #f093fb 0%, #f5576c 100%)',  
    solid: '#f5576c',  
    light: '#fbb6ce'  
  },  
  success: '#48bb78',  
  warning: '#f6ad55',  
  error: '#fc8181',  
  neutral: {  
    bg: '#f7fafc',  
    card: 'ffffff',  
    text: '#2d3748',  
    textLight: '#718096',  
    border: '#e2e8f0'  
  }  
}
```



Tipografia

typescript

```
// theme/typography.ts
```

```
export const typography = {  
  fontFamily: "'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif",  
  sizes: {  
    h1: '32px',    // Dashboard titles  
    h2: '24px',    // Section titles  
    h3: '18px',    // Card titles  
    body: '16px',  // Regular text  
    small: '14px', // Secondary text  
    xs: '12px'     // Labels, badges  
  },  
  weights: {  
    regular: 400,  
    medium: 500,  
    semibold: 600,  
    bold: 700  
  }  
}
```



Breakpoints (Responsive)

```
typescript
```

```
// theme/breakpoints.ts
```

```
export const breakpoints = {  
  mobile: '320px',  
  tablet: '768px',  
  desktop: '1024px',  
  wide: '1440px'  
}
```

```
// Tailwind config
```

```
module.exports = {  
  theme: {  
    screens: {  
      'sm': '640px',  
      'md': '768px',  
      'lg': '1024px',  
      'xl': '1280px',  
      '2xl': '1536px',  
    }  
  }  
}
```

Animaciones y Transiciones

CSS

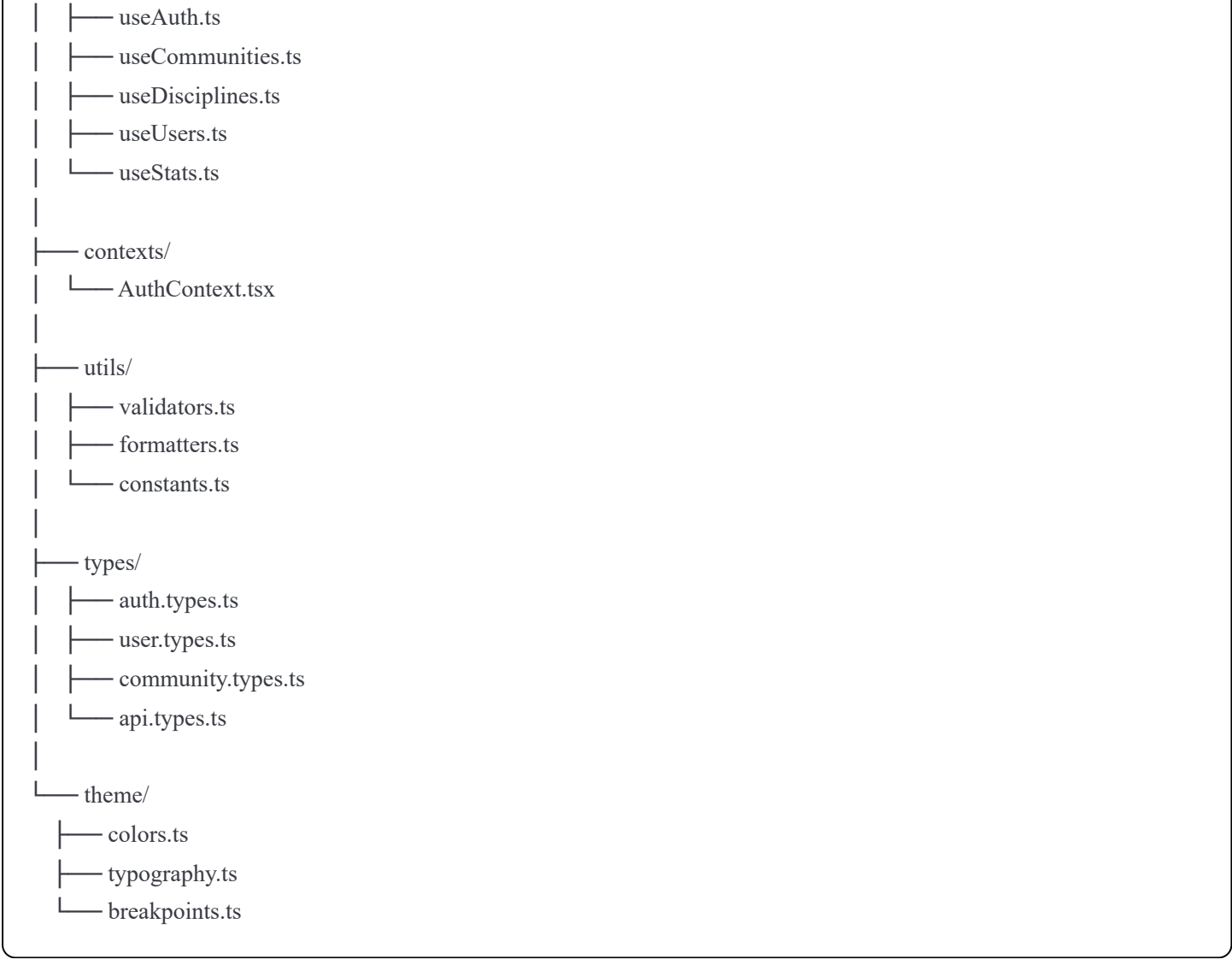
```
/* Transiciones suaves */  
.smooth-transition {  
  transition: all 0.3s ease-in-out;  
}  
  
/* Hover effects */  
.card-hover:hover {  
  transform: translateY(-5px);  
  box-shadow: 0 12px 32px rgba(102, 126, 234, 0.3);  
}  
  
/* Loading states */  
@keyframes pulse {  
  0%, 100% { opacity: 1; }  
  50% { opacity: 0.5; }  
}
```

Mapeo de Componentes React

Estructura de Carpetas

```
src/  
├── api/  
│   ├── client.ts          # Axios instance  
│   ├── generated/         # OpenAPI generated  
│   └── services/  
│       ├── auth.service.ts  
│       ├── user.service.ts  
│       ├── community.service.ts  
│       ├── discipline.service.ts  
│       └── stats.service.ts  
├── components/  
│   ├── common/  
│   │   ├── Button.tsx  
│   │   ├── Input.tsx  
│   │   ├── Card.tsx  
│   │   ├── Modal.tsx  
│   │   ├── LoadingSpinner.tsx  
│   │   └── ErrorBoundary.tsx  
│   └── ...  
└── ...
```

```
| | — layout/
| | | — NavBar.tsx
| | | — Sidebar.tsx
| | | — Footer.tsx
| | | — DashboardLayout.tsx
| |
| | — auth/
| | | — LoginForm.tsx
| | | — RegisterForm.tsx
| | | — AuthProvider.tsx
| |
| | — community/
| | | — CommunityCard.tsx
| | | — CommunityList.tsx
| | | — CommunityForm.tsx
| | | — CommunityTable.tsx
| | | — MembersList.tsx
| |
| | — discipline/
| | | — DisciplineCard.tsx
| | | — DisciplineGrid.tsx
| | | — DisciplineForm.tsx
| |
| | — user/
| | | — UserProfile.tsx
| | | — UserTable.tsx
| | | — AvatarUpload.tsx
| | | — ProfileForm.tsx
| |
| | — stats/
| | | — StatsCards.tsx
| | | — RankingChart.tsx
| | | — DistributionChart.tsx
| | | — SummaryMetrics.tsx
| |
| — pages/
| | — LoginPage.tsx
| | — RegisterPage.tsx
| | — DashboardPage.tsx
| | — ProfilePage.tsx
| | — CommunitiesPage.tsx
| | — CommunityDetailPage.tsx
| | — DisciplinesPage.tsx
| | — UsersPage.tsx
| | — StatsPage.tsx
| |
| — hooks/
```



Componentes Clave y Responsabilidades

Componente	Responsabilidad	Props Principales
AuthProvider	Manejo de autenticación global	children
useAuth	Hook para acceder al contexto auth	-
NavBar	Navegación principal	user, onLogout
DashboardLayout	Layout principal con sidebar	children
CommunityCard	Tarjeta de comunidad	community, onJoin
CommunityForm	Formulario CRUD comunidades	community?, onSubmit
StatsCards	Tarjetas de estadísticas	stats
RankingChart	Gráfico de ranking (Recharts)	data

Integración con API

Configuración del Cliente HTTP

typescript

```
// src/api/client.ts
import axios from 'axios';

const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:8000/api/v1',
  headers: {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
  },
});

// Interceptor para añadir token
apiClient.interceptors.request.use((config) => {
  const token = localStorage.getItem('token'); // O desde contexto
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Interceptor para manejar errores
apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      // Redirigir a login
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

export default apiClient;
```

Servicios por Módulo

typescript

```
// src/api/services/auth.service.ts
```

```
import apiClient from '../client';

export const authService = {
  login: async (email: string, password: string) => {
    const { data } = await apiClient.post('/login', { email, password });
    return data;
  },

  register: async (userData: RegisterData) => {
    const { data } = await apiClient.post('/register', userData);
    return data;
  },

  logout: async () => {
    const { data } = await apiClient.post('/logout');
    return data;
  },

  me: async () => {
    const { data } = await apiClient.get('/me');
    return data;
  }
};
```

Hooks con TanStack Query

typescript

```
// src/hooks/useCommunities.ts
```

```
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
```

```
import { communityService } from '@api/services/community.service';
```

```
export const useCommunities = () => {
```

```
  return useQuery({
```

```
    queryKey: ['communities'],
```

```
    queryFn: communityService.getAll,
```

```
  });
```

```
};
```

```
export const useCreateCommunity = () => {
```

```
  const queryClient = useQueryClient();
```

```
  return useMutation({
```

```
    mutationFn: communityService.create,
```

```
    onSuccess: () => {
```

```
      queryClient.invalidateQueries({ queryKey: ['communities'] });
```

```
    },
```

```
  });
```

```
};
```



Plan de Implementación

Fase 1: Setup y Auth (Sprint 1 - 1 semana)

Tareas:

- ✓ Inicializar proyecto Vite + TypeScript
- ✓ Configurar TailwindCSS
- ✓ Generar cliente API desde OpenAPI
- ✓ Implementar AuthProvider y useAuth
- ✓ Crear LoginPage y RegisterPage
- ✓ Configurar React Router
- ✓ Implementar ProtectedRoute

Entregables:

- Login/Register funcionales
- Autenticación con Bearer token
- Navegación básica

Fase 2: Dashboard y Componentes Core (Sprint 2 - 1 semana)

Tareas:

-  Crear DashboardLayout
-  Implementar NavBar con menú de usuario
-  Desarrollar StatsCards
-  Crear CommunityList y DisciplineGrid
-  Implementar ProfilePage

Entregables:

- Dashboard funcional
- Vista de perfil
- Navegación completa

Fase 3: CRUD y Gestión (Sprint 3 - 1 semana)

Tareas:

-  Implementar CommunitiesPage con CRUD
-  Crear DisciplinesPage (admin)
-  Desarrollar UsersPage (admin/mod)
-  Implementar formularios y validaciones
-  Añadir confirmaciones de eliminación

Entregables:

- CRUD completo de comunidades
- Gestión de disciplinas (admin)
- Gestión de usuarios (admin/mod)

Fase 4: Estadísticas y Visualizaciones (Sprint 4 - 1 semana)

Tareas:

-  Integrar Recharts
-  Crear RankingChart
-  Desarrollar DisciplineDistributionChart
-  Implementar StatsPage completa
-  Añadir filtros y exportación

Entregables:

- Dashboard de estadísticas
- Gráficos interactivos
- Exportación de datos

Fase 5: Testing y Optimización (Sprint 5 - 1 semana)

Tareas:

- ☒ Tests unitarios (Vitest)
- ☒ Tests E2E (Playwright)
- ☒ Optimización de rendimiento
- ☒ Implementar lazy loading
- ☒ Code splitting
- ☒ Optimizar imágenes y assets

Entregables:

- Suite de tests completa
- Aplicación optimizada
- Documentación técnica

Fase 6: Docker y Deployment (Sprint 6 - 3 días)

Tareas:

- ☒ Crear Dockerfile para frontend
- ☒ Configurar docker-compose
- ☒ Setup CI/CD (GitHub Actions)
- ☒ Deployment a producción
- ☒ Configurar variables de entorno

Entregables:

- Aplicación dockerizada
 - Pipeline CI/CD funcional
 - Deployment automatizado
-



Testing Strategy

Tests Unitarios (Vitest)

typescript

```
// src/components/auth/LoginForm.test.tsx
```

```
import { render, screen, fireEvent, waitFor } from '@testing-library/react';
import { describe, it, expect, vi } from 'vitest';
import LoginForm from './LoginForm';

describe('LoginForm', () => {
  it('renders login form correctly', () => {
    render(<LoginForm onSubmit={vi.fn()} />);

    expect(screen.getByLabelText(/email/i)).toBeInTheDocument();
    expect(screen.getByLabelText(/password/i)).toBeInTheDocument();
    expect(screen.getByRole('button', { name: /iniciar sesión/i })).toBeInTheDocument();
  });

  it('validates email format', async () => {
    const onSubmit = vi.fn();
    render(<LoginForm onSubmit={onSubmit} />);

    const emailInput = screen.getByLabelText(/email/i);
    const submitButton = screen.getByRole('button', { name: /iniciar sesión/i });

    fireEvent.change(emailInput, { target: { value: 'invalid-email' } });
    fireEvent.click(submitButton);

    await waitFor(() => {
      expect(screen.getByText(/email inválido/i)).toBeInTheDocument();
    });

    expect(onSubmit).not.toHaveBeenCalled();
  });

  it('submits form with valid data', async () => {
    const onSubmit = vi.fn();
    render(<LoginForm onSubmit={onSubmit} />);

    fireEvent.change(screen.getByLabelText(/email/i), {
      target: { value: 'doom@user.com' }
    });
    fireEvent.change(screen.getByLabelText(/password/i), {
      target: { value: '666' }
    });
    fireEvent.click(screen.getByRole('button', { name: /iniciar sesión/i }));

    await waitFor(() => {
      expect(onSubmit).toHaveBeenCalledWith({
        email: 'doom@user.com',

```

```
    password: '666'  
  });  
});  
});  
});
```

Tests de Integración

typescript

```
// src/hooks/useAuth.test.ts  
  
import { renderHook, waitFor } from '@testing-library/react';  
import { describe, it, expect, vi } from 'vitest';  
import { useAuth } from './useAuth';  
import { authService } from '@api/services/auth.service';  
  
vi.mock('@api/services/auth.service');  
  
describe('useAuth', () => {  
  it('logs in user successfully', async () => {  
    const mockToken = 'mock-token-123';  
    const mockUser = { id: 1, email: 'doom@user.com', role: 'User' };  
  
    vi.mocked(authService.login).mockResolvedValue({  
      token: mockToken,  
      user: mockUser  
    });  
  
    const { result } = renderHook(() => useAuth());  
  
    await waitFor(() => {  
      result.current.login('doom@user.com', '666');  
    });  
  
    expect(result.current.user).toEqual(mockUser);  
    expect(result.current.isAuthenticated).toBe(true);  
  });  
});
```

Tests E2E (Playwright)

typescript

```
// e2e/auth.spec.ts
```

```
import { test, expect } from '@playwright/test';

test.describe('Authentication Flow', () => {
  test('should login successfully', async ({ page }) => {
    await page.goto('http://localhost:5173/login');

    await page.fill('input[name="email"]', 'doom@user.com');
    await page.fill('input[name="password"]', '666');
    await page.click('button[type="submit"]');

    await expect(page).toHaveURL('http://localhost:5173/');
    await expect(page.locator('text=Hola, Marc Sanchez')).toBeVisible();
  });

  test('should show error with invalid credentials', async ({ page }) => {
    await page.goto('http://localhost:5173/login');

    await page.fill('input[name="email"]', 'invalid@user.com');
    await page.fill('input[name="password"]', 'wrong');
    await page.click('button[type="submit"]');

    await expect(page.locator('text=/credenciales inválidas/i')).toBeVisible();
  });
});
```

Seguridad

Buenas Prácticas Implementadas

1. Autenticación:

- Tokens Bearer en header Authorization
- Refresh token strategy (si disponible en API)
- Logout que invalida token en servidor

2. Almacenamiento:

- Token en memoria durante sesión activa
- Cookie httpOnly para refresh token (recomendado)
- No almacenar datos sensibles en localStorage

3. Validación:

- Validación en cliente Y servidor
- Sanitización de inputs

- Protección contra XSS

4. Permisos:

- Rutas protegidas con `ProtectedRoute`
- Verificación de roles en componentes
- Ocultación de acciones no permitidas

5. HTTPS:

- Obligatorio en producción
- Configuración de CORS correcta
- CSP headers

Implementación de `ProtectedRoute`

typescript

```
// src/components/auth/ProtectedRoute.tsx
```

```
import { Navigate } from 'react-router-dom';
import { useAuth } from '@/hooks/useAuth';
```

```
interface ProtectedRouteProps {
  children: React.ReactNode;
  allowedRoles?: string[];
}
```

```
export const ProtectedRoute: React.FC<ProtectedRouteProps> = ({
  children,
  allowedRoles
}) => {
  const { isAuthenticated, user } = useAuth();
```

```
  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }
```

```
  if (allowedRoles && !allowedRoles.includes(user.role)) {
    return <Navigate to="/" replace />;
  }
```

```
  return <>{children}</>;
};
```

Docker Configuration

Dockerfile

```
dockerfile

# Dockerfile
FROM node:20-alpine AS builder

WORKDIR /app

# Copiar package.json y package-lock.json
COPY package*.json ./

# Instalar dependencias
RUN npm ci

# Copiar código fuente
COPY . .

# Build de producción
RUN npm run build

# Stage de producción con nginx
FROM nginx:alpine

# Copiar build al directorio de nginx
COPY --from=builder /app/dist /usr/share/nginx/html

# Copiar configuración personalizada de nginx
COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

docker-compose.yml

```
yaml
```

version: '3.8'

services:

Frontend React

frontend:

build:

context: .

dockerfile: Dockerfile

ports:

- "3000:80"

environment:

- VITE_API_URL=http://localhost:8000/api/v1

depends_on:

- backend

networks:

- fitbit-network

Backend Laravel (opcional, si también dockerizas backend)

backend:

image: fitbit-api:latest

ports:

- "8000:8000"

environment:

- DB_HOST=mysql

- DB_DATABASE=fitbit_api

- DB_USERNAME=fitbit_user

- DB_PASSWORD=fitbit_pass

depends_on:

- mysql

networks:

- fitbit-network

Base de datos MySQL

mysql:

image: mysql:8

ports:

- "3306:3306"

environment:

- MYSQL_ROOT_PASSWORD=root

- MYSQL_DATABASE=fitbit_api

- MYSQL_USER=fitbit_user

- MYSQL_PASSWORD=fitbit_pass

volumes:

- mysql-data:/var/lib/mysql

networks:

- fitbit-network

```
volumes:
  mysql-data:

networks:
  fitbit-network:
    driver: bridge
```

nginx.conf

```
nginx

server {
  listen 80;
  server_name localhost;
  root /usr/share/nginx/html;
  index index.html;

  location / {
    try_files $uri $uri/ /index.html;
  }

  # Gzip compression
  gzip on;
  gzip_vary on;
  gzip_min_length 1024;
  gzip_types text/plain text/css text/xml text/javascript
    application/x-javascript application/xml+rss
    application/javascript application/json;
}
```

CI/CD Pipeline

GitHub Actions

```
yaml
```

.github/workflows/ci-cd.yml

name: CI/CD Pipeline

on:

push:

branches: [main, develop]

pull_request:

branches: [main]

jobs:

test:

runs-on: ubuntu-latest

steps:

- **uses:** actions/checkout@v3

- **name:** Setup Node.js

uses: actions/setup-node@v3

with:

node-version: '20'

cache: 'npm'

- **name:** Install dependencies

run: npm ci

- **name:** Run linter

run: npm run lint

- **name:** Run tests

run: npm run test

- **name:** Build

run: npm run build

env:

VITE_API_URL: \${ secrets.VITE_API_URL }

deploy:

needs: test

runs-on: ubuntu-latest

if: github.ref == 'refs/heads/main'

steps:

- **uses:** actions/checkout@v3

- **name:** Setup Node.js

uses: actions/setup-node@v3

with:

node-version: '20'

- name: Install dependencies

run: npm ci

- name: Build

run: npm run build

env:

VITE_API_URL: \${ secrets.VITE_API_URL }

- name: Deploy to Vercel

uses: amondnet/vercel-action@v20

with:

vercel-token: \${ secrets.VERCEL_TOKEN }

vercel-org-id: \${ secrets.VERCEL_ORG_ID }

vercel-project-id: \${ secrets.VERCEL_PROJECT_ID }

Documentación de Componentes

Ejemplo: CommunityCard

typescript

```

/**
 * CommunityCard Component
 *
 * Muestra una tarjeta con información de una comunidad
 *
 * @param {Community} community - Objeto con datos de la comunidad
 * @param {boolean} showActions - Mostrar botones de acción (default: true)
 * @param {Function} onJoin - Callback al unirse a comunidad
 * @param {Function} onLeave - Callback al salir de comunidad
 * @param {Function} onEdit - Callback para editar (solo admin/mod)
 * @param {Function} onDelete - Callback para eliminar (solo admin/mod)
 *
 * @example
 * <CommunityCard
 *   community={community}
 *   onJoin={handleJoin}
 *   onLeave={handleLeave}
 * />
 */
interface CommunityCardProps {
  community: Community;
  showActions?: boolean;
  onJoin?: (id: number) => void;
  onLeave?: (id: number) => void;
  onEdit?: (id: number) => void;
  onDelete?: (id: number) => void;
}

export const CommunityCard: React.FC<CommunityCardProps> = ({
  community,
  showActions = true,
  onJoin,
  onLeave,
  onEdit,
  onDelete
}) => {
  // Implementation...
};

```

Métricas de Performance

Objetivos

Métrica	Target	Actual
First Contentful Paint	< 1.5s	TBD
Largest Contentful Paint	< 2.5s	TBD
Time to Interactive	< 3.5s	TBD
Cumulative Layout Shift	< 0.1	TBD
Bundle Size (gzipped)	< 200KB	TBD

Optimizaciones

1. Code Splitting:

```
typescript

// Lazy loading de rutas
const DashboardPage = lazy(() => import('@pages/DashboardPage'));
const CommunitiesPage = lazy(() => import('@pages/CommunitiesPage'));
```

2. Image Optimization:

- Formato WebP
- Lazy loading de imágenes
- Responsive images

3. Caching:

- TanStack Query cache
- Service Worker (PWA)

Checklist Pre-Launch

Desarrollo

- ☐ Todos los endpoints integrados
- ☐ Validaciones implementadas
- ☐ Manejo de errores completo
- ☐ Loading states en todas las acciones
- ☐ Responsive design verificado

Testing

- ☐ Tests unitarios > 80% coverage

- ☐ Tests E2E críticos pasando
- ☐ Cross-browser testing (Chrome, Firefox, Safari)
- ☐ Testing en dispositivos móviles

Seguridad

- ☐ HTTPS configurado
- ☐ Tokens seguros
- ☐ Validación de roles implementada
- ☐ XSS y CSRF protección
- ☐ Rate limiting en cliente

Performance

- ☐ Lighthouse score > 90
- ☐ Bundle size optimizado
- ☐ Lazy loading implementado
- ☐ Images optimizadas

Deployment

- ☐ Variables de entorno configuradas
- ☐ Docker build exitoso
- ☐ CI/CD pipeline funcionando
- ☐ Backup strategy definida
- ☐ Monitoring configurado

Soporte y Contacto

Documentación adicional:

- API Swagger: <http://localhost:8000/api/documentation>
- Storybook (opcional): <http://localhost:6006>

Repositorio:

- Frontend: github.com/usuario/fitbit_frontend
- Backend: github.com/usuario/fitbit_api

Versionado

- **v1.0.0** - MVP inicial (Enero 2025)
- **v1.1.0** - Features adicionales (Febrero 2025)

- **v2.0.0** - Refactor y optimizaciones (Marzo 2025)
-

Notas Finales

Este diseño UX/UI está preparado para:

-  Integración inmediata con Laravel API
-  Desarrollo ágil con componentes modulares
-  Escalabilidad futura
-  Mantenimiento sencillo
-  Experiencia de usuario óptima

Próximo paso: Implementación de componentes React según este wireframe.

Fecha de creación: Enero 2025

Última actualización: Enero 2025

Autor: React Frontend Assistant